Universidad San Carlos de Guatemala.

Centro Universitario de Occidente.

Curso: Sistemas de Bases de Datos 2.

Catedrático: Daniel Gonzalez.

Sección: A.

Manual Técnico

Nombre: Miguel Angel Quemé Gómez

Carné: 202031398

Quetzaltenango, 30 de octubre del 2026.

Índice

1. Introducción 2
1.1 Objetivos del sistema 2
1.2 Tecnologías utilizadas 2
2. Arquitectura 3
2.1 Responsabilidades de cada capa 3
2.2 Estructura del código 4
3. Instalación y configuración 5
3.1 Requisitos 5
3.2 Despliegue con Docker Compose 5
3.3 Variables de configuración 6
3.4 Modo de desarrollo 6
4. Modelo documental (MongoDB) 7
4.1 Colección evaluaciones 7
4.2 Colecciones derivadas y de control 8
4.3 Índices 8
5. Proceso ETL 9
5.1 Etapas 9
5.2 Reglas de limpieza y estandarización 10
5.3 Particularidades de los archivos 2022–2024 11
5.4 Resultado de la carga del dataset 11
6. Modelo de grafos (Neo4j) 13
7. API REST 15
8. Manejo de errores 16
9. Ventajas y limitaciones de usar MongoDB y Neo4j 17

1. Introducción

Este manual describe la arquitectura, la configuración, los modelos de datos, el proceso de carga (ETL), la API y el manejo de errores de la aplicación **Pruebas Específicas USAC**. La aplicación carga los archivos CSV del conjunto de datos *Pruebas específicas* del portal de Datos Abiertos de la Facultad de Ingeniería, los valida y transforma, los almacena en **MongoDB** (base documental) y en **Neo4j** (base de grafos) y presenta los resultados en una interfaz web.

1.1 Objetivos del sistema

- Cargar uno o varios archivos CSV desde la interfaz o desde la terminal, con validación y registro de los registros rechazados y su causa.
- Almacenar las evaluaciones en MongoDB y resolver las 20 consultas analíticas solicitadas con el *Aggregation Framework*.
- Representar en Neo4j las relaciones entre aspirantes, materias, carreras, instituciones, municipios y departamentos, y explorarlas con consultas Cypher.
- Visualizar estadísticas, grafos y búsquedas desde una interfaz que nunca se conecta directamente a las bases de datos.

1.2 Tecnologías utilizadas

Tabla 1. Tecnologías del sistema

Componente	Tecnología	Uso en el proyecto
Frontend	React 18 + Vite 6	Interfaz de una sola página (SPA): dashboard, explorador de grafos, buscador, carga y consola
Gráficas	Recharts 2 y react-force-graph-2d	Gráficas de barras, pastel y líneas; dibujo interactivo de grafos
Servidor web	nginx 1.27	Sirve el frontend compilado y reenvía `/api` al backend
Backend / API	Node.js 22 + Express 5	API REST, validación de parámetros, ETL, ejecución de consultas y manejo de errores
Lectura de CSV	csv-parse 7 (streaming) y multer 2	Recepción de archivos y lectura fila por fila sin cargar todo en memoria
Base documental	MongoDB 7.0 (driver mongodb 7)	Evaluaciones, colecciones derivadas y consultas analíticas
Base de grafos	Neo4j 5.26 (neo4j-driver 6)	Nodos y relaciones; consultas Cypher y recorridos
Contenedores	Docker Compose	Levanta los cuatro servicios con un solo comando
Diagramas	draw.io	Diagramas de este manual (archivos `.drawio` en `docs/diagramas/`)

2. Arquitectura

La solución sigue una arquitectura de tres capas con una separación clara entre la interfaz, la lógica de aplicación y las bases de datos. El navegador sólo se comunica con nginx; nginx entrega los archivos estáticos del frontend y reenvía las rutas `/api` al backend, que es el único componente que se conecta a MongoDB y Neo4j.

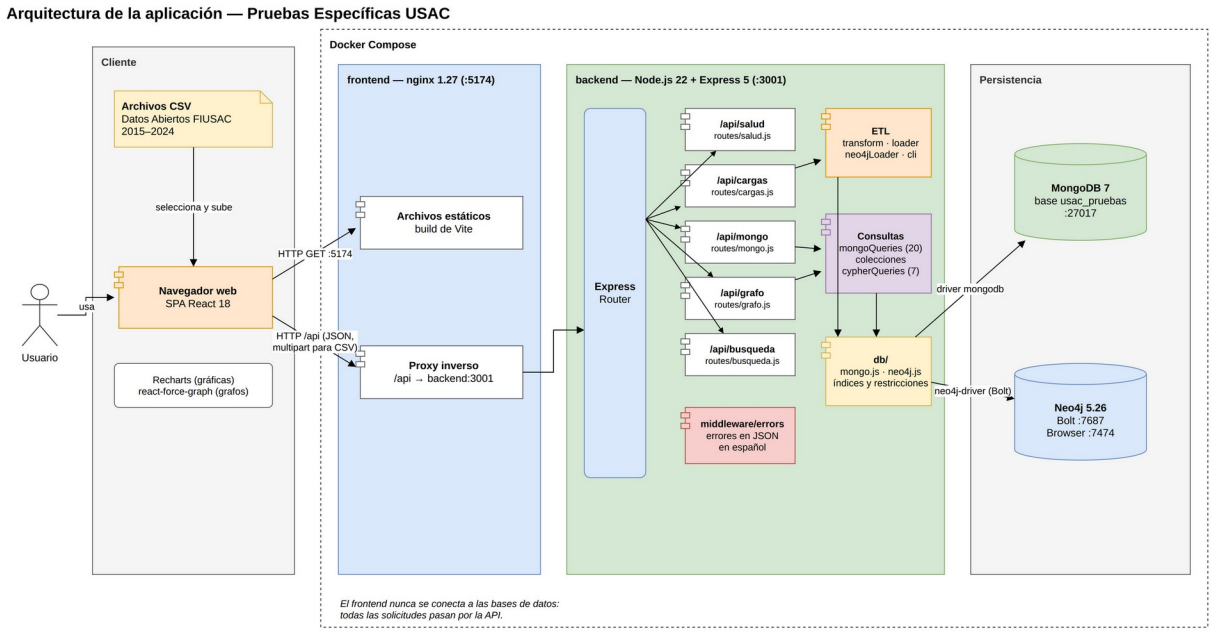


Figura 1. *Diagrama de arquitectura de la aplicación*

2.1 Responsabilidades de cada capa

Tabla 2. Responsabilidades por capa

Capa	Responsabilidades
Frontend (React)	Mostrar el dashboard, los grafos, el buscador y la carga de archivos; validar entradas del usuario; mostrar errores legibles sin cerrar la aplicación (`ErrorBoundary`).
Proxy (nginx)	Servir el *build* de Vite; reenviar `/api` al backend; permitir archivos de hasta 500 MB y cargas de hasta una hora.
Backend (Express)	Recibir las solicitudes, validar parámetros, ejecutar el ETL, ejecutar consultas en MongoDB y Neo4j, transformar resultados y responder errores en JSON con mensajes en español.
MongoDB	Guardar las evaluaciones limpias, las colecciones derivadas (`aspirantes`, `resumen_carrera`) y las colecciones de control (`cargas`, `registros_rechazados`).
Neo4j	Guardar el grafo de aspirantes, materias, carreras, instituciones y ubicaciones para consultas de relaciones y recorridos.

2.2 Estructura del código

El repositorio separa el backend y el frontend en carpetas independientes. Dentro de cada una, el código se organiza por responsabilidad, como muestra el diagrama de paquetes.

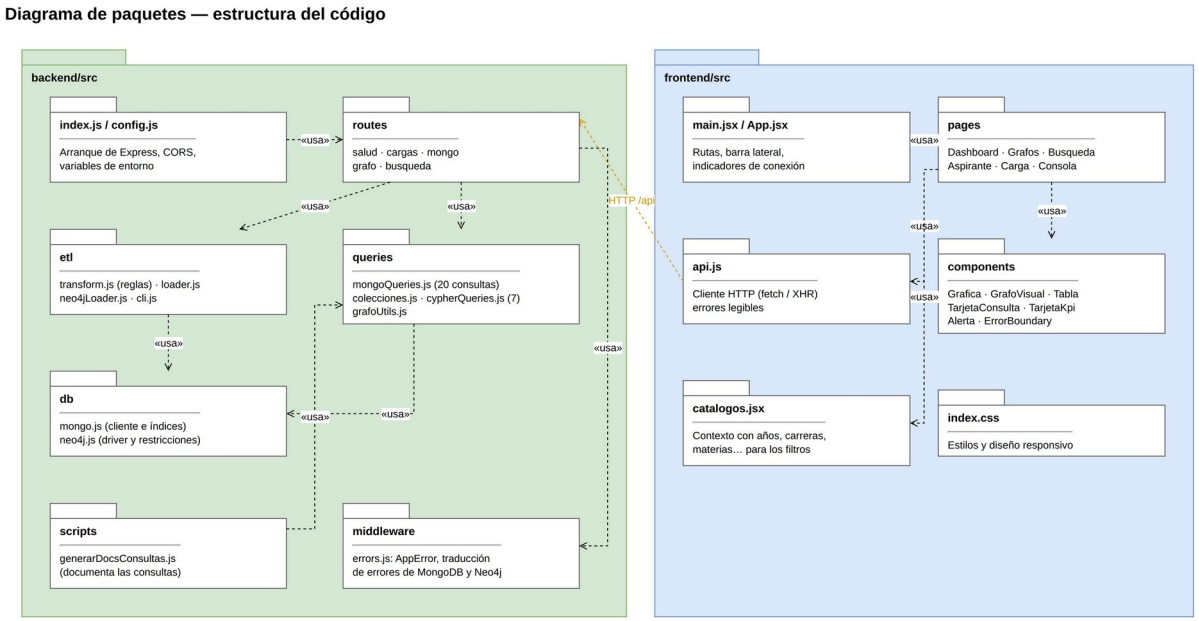


Figura 2. *Diagrama de paquetes del backend y del frontend*

Tabla 3. Carpetas principales del repositorio

Ruta	Contenido
`docker-compose.yml`	Servicios mongo, neo4j, backend y frontend
`backend/src/index.js`	Arranque de Express, registro de rutas y cierre ordenado de conexiones
`backend/src/db/`	`mongo.js` (cliente e índices) y `neo4j.js` (driver y restricciones de unicidad)
`backend/src/etl/`	`transform.js` (reglas de limpieza), `loader.js` (proceso de carga), `neo4jLoader.js` (MERGE en el grafo), `cli.js` (carga desde terminal)
`backend/src/queries/`	`mongoQueries.js` (20 consultas), `colecciones.js` (colecciones derivadas), `cypherQueries.js` (7 consultas), `grafoUtils.js`
`backend/src/routes/`	Rutas `salud`, `cargas`, `mongo`, `grafo` y `busqueda`
`backend/src/middleware/errors.js`	Clase `AppError` y traducción de errores de MongoDB y Neo4j
`frontend/src/pages/`	Dashboard, Grafos, Busqueda, Aspirante, Carga y Consola
`frontend/src/components/`	Grafica, GrafoVisual, Tabla, TarjetaConsulta, TarjetaKpi, Alerta y ErrorBoundary
`consultas/`	Consultas listas para ejecutar en mongosh y en Neo4j Browser
`docs/`	Manuales, diagramas `.drawio`, capturas y documentación de consultas

3. Instalación y configuración

3.1 Requisitos

- Docker y Docker Compose.
- Node.js 20.19 o superior (sólo para el modo de desarrollo sin contenedores).
- Los archivos CSV del dataset en la carpeta `data/`.

3.2 Despliegue con Docker Compose

Todo el sistema se levanta con un solo comando desde la raíz del proyecto. El backend espera a que MongoDB y Neo4j estén sanos (healthchecks) antes de iniciar, de modo que siempre crea los índices y las restricciones al arrancar.

```
docker compose up -d --build
```

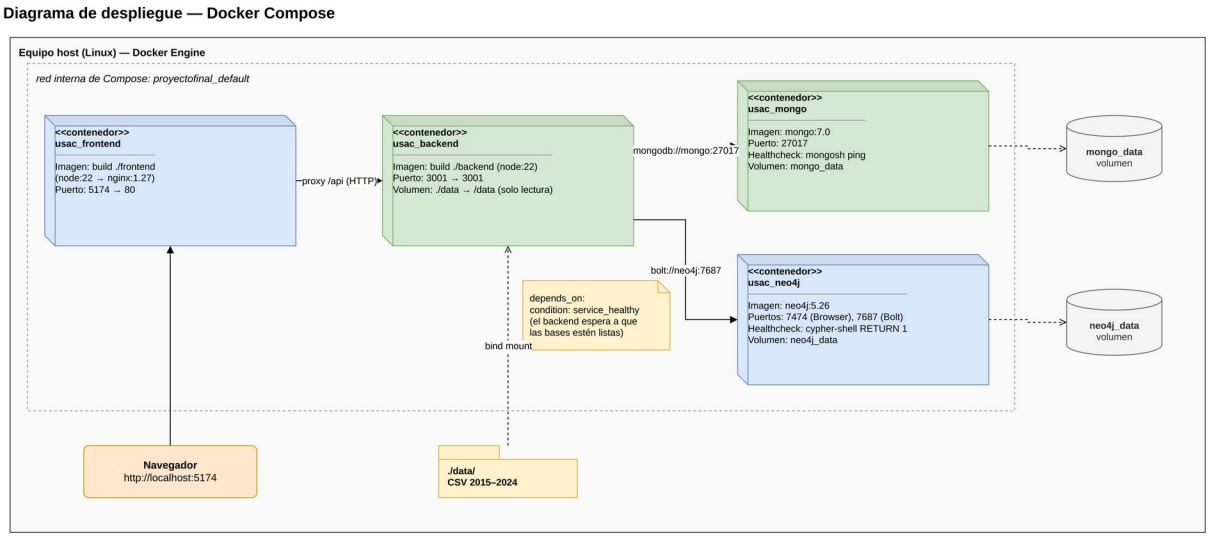


Figura 3. *Diagrama de despliegue con Docker Compose*

Tabla 4. Contenedores del sistema

Servicio	Imagen	Puertos	Persistencia
usac_frontend	build ./frontend (node:22 → nginx:1.27)	5174 → 80	—
usac_backend	build ./backend (node:22-alpine)	3001	`./data` montada en `/data` (solo lectura)
usac_mongo	mongo:7.0	27017	volumen `mongo_data`
usac_neo4j	neo4j:5.26	7474 (Browser), 7687 (Bolt)	volumen `neo4j_data`

Comandos útiles:

Tabla 5. Comandos de operación

Acción	Comando
Aplicar cambios de código	`docker compose up -d --build` (o sólo `backend` / `frontend`)
Ver el estado de los contenedores	`docker compose ps`
Ver los registros del backend	`docker compose logs -f backend`
Cargar todos los CSV desde la terminal	`docker compose exec backend sh -c 'npm run cargar -- /data/pruebas_especificas_20*.csv'`
Detener sin borrar datos	`docker compose down` (no usar `-v`, que elimina los volúmenes)

3.3 Variables de configuración

El backend lee su configuración de variables de entorno (en Docker se definen en `docker-compose.yml`; en desarrollo, en `backend/.env` a partir de `.env.example`).

Tabla 6. Variables de entorno del backend

Variable	Valor en Docker	Descripción
`PORT`	3001	Puerto de la API
`MONGO_URI`	mongodb://admin:admin123@mongo:27017/?authSource=admin	Cadena de conexión a MongoDB
`MONGO_DB`	usac_pruebas	Nombre de la base de datos
`NEO4J_URI`	bolt://neo4j:7687	URI Bolt de Neo4j
`NEO4J_USER` / `NEO4J_PASSWORD`	neo4j / password123	Credenciales de Neo4j
`BATCH_SIZE`	2000	Registros por lote al insertar en ambas bases
`CORS_ORIGIN`	* (no se usa detrás de nginx)	Origen permitido en modo de desarrollo

3.4 Modo de desarrollo

Para modificar código con recarga automática se levantan sólo las bases en Docker y el backend y el frontend con npm:

```
docker compose up -d mongo neo4j
cd backend && cp .env.example .env && npm install && npm run dev
cd frontend && npm install && npm run dev
```

En este modo, Vite reenvía `/api` a `http://localhost:3001` mediante su *proxy*, por lo que el frontend tampoco se conecta a las bases de datos.

4. Modelo documental (MongoDB)

La base `usac_pruebas` contiene una colección principal con una evaluación por documento, dos colecciones derivadas que se regeneran después de cada carga y dos colecciones de control del proceso de carga.

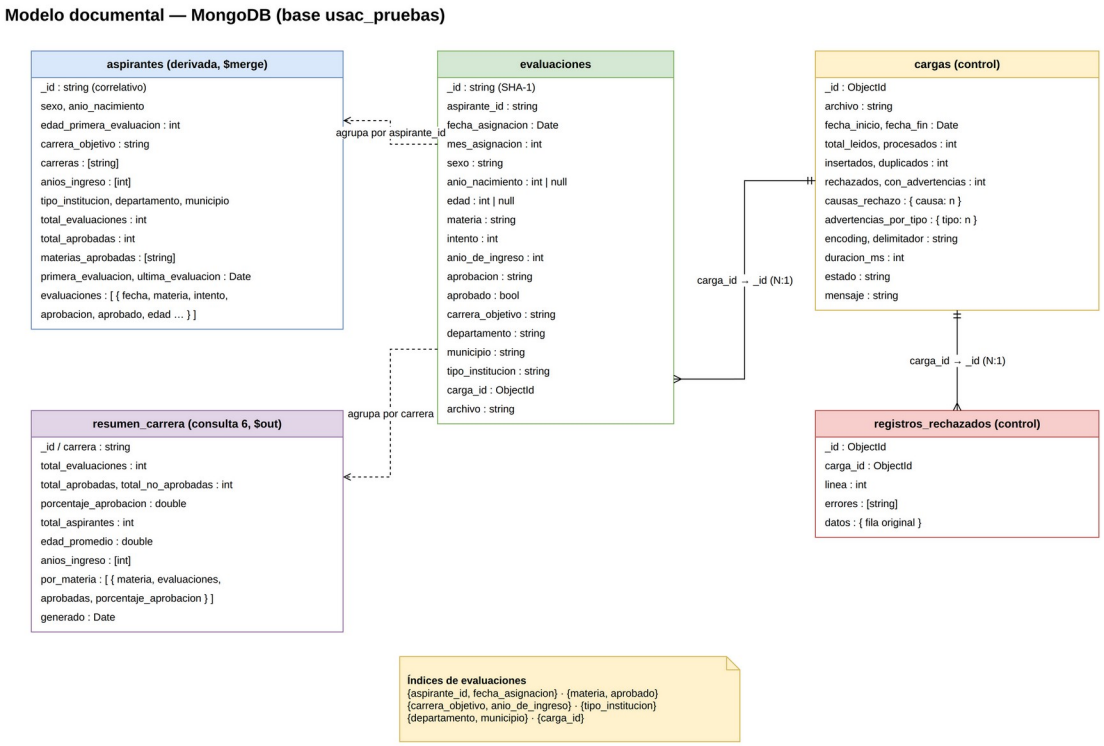


Figura 4. *Modelo documental de MongoDB*

4.1 Colección evaluaciones

Cada fila válida del CSV se guarda como un documento ya limpio y tipado:

```
{
  "_id": "sha1(aspirante|fecha|materia|intento|anio_ingreso|aprobacion)",
  "aspirante_id": "4cdb093179e6dba32f813f85041b8274",
  "fecha_asignacion": ISODate("2017-08-28"),
  "mes_asignacion": 8,
  "sexo": "M",
  "anio_nacimiento": 1996,
  "edad": 21,
  "materia": "MATEMATICA",
  "intento": 1,
  "anio_de_ingreso": 2018,
  "aprobacion": "NO APROBADO",
  "aprobado": false,
  "carrera_objetivo": "INGENIERIA EN CIENCIAS Y SISTEMAS",
  "departamento": "QUEZALTENANGO",
  "municipio": "QUETZALTENANGO",
  "tipo_institucion": "PRIVADO",
  "carga_id": ObjectId("..."),
  "archivo": "pruebas_especificas_2018.csv"
}
```

Decisiones de diseño:

- **Una evaluación por documento.** Es la unidad natural del dataset y permite agrupar por cualquier campo con `$group`. Como un aspirante aparece en varias filas, los conteos de aspirantes se hacen con correlativos distintos, agrupando primero por `aspirante_id`.
- **`_id` determinístico.** Es el hash SHA-1 de los campos que identifican la evaluación. Si un archivo se carga dos veces, o trae filas repetidas, MongoDB rechaza la clave duplicada y el registro se cuenta como duplicado sin insertarse otra vez.
- **Campos derivados precalculados** en el ETL (`edad`, `aprobado`, `mes_asignacion`) para simplificar y acelerar las consultas.

4.2 Colecciones derivadas y de control

Tabla 7. Colecciones derivadas y de control

Colección	Cómo se genera	Para qué se usa
`aspirantes`	`$group` por aspirante y `$merge` al terminar cada carga	Un documento por aspirante con su historial embebido; resuelve la consulta 15 con una sola lectura
`resumen_carrera`	`$group` por carrera y `$out` (consulta 6)	Totales, porcentaje de aprobación, aspirantes, edad promedio y desglose por materia
`cargas`	Un documento por archivo procesado	Historial de cargas: leídos, procesados, insertados, duplicados, rechazados, causas, tiempo y estado
`registros_rechazados`	Una fila rechazada por documento	Número de línea, lista de errores y datos originales de cada registro rechazado

4.3 Índices

Tabla 8. Índices de la colección evaluaciones

Índice en evaluaciones	Consultas que lo aprovechan
`{aspirante_id: 1, fecha_asignacion: 1}`	Historial de un aspirante (11, 15), buscador por correlativo
`{materia: 1, aprobado: 1}`	Aprobados y porcentaje por materia (2, 4, 14)
`{carrera_objetivo: 1, anio_de_ingreso: 1}`	Consultas por carrera y año (3, 5, 16, 20)
`{tipo_institucion: 1}`	Consultas por tipo de institución (1, 9, 12, 18)
`{departamento: 1, municipio: 1}`	Consultas por ubicación (8, 17, 19) y buscador
`{carga_id: 1}`	Consultas por carga y limpieza de datos

Ejemplo de optimización: el buscador filtraba el departamento con una expresión regular sin `^`, lo que obligaba a MongoDB a revisar las 207,619 entradas del índice (144 ms según `explain()`). Al usar igualdad exacta para los campos que vienen de una lista, el índice `{departamento, municipio}` va directo a las entradas del departamento y la misma búsqueda tarda 1 ms.

5. Proceso ETL

El proceso de extracción, transformación y carga está en `backend/src/etl/`. Se ejecuta desde la interfaz (`POST /api/cargas`) o desde la terminal (`npm run cargar`). Cada archivo se procesa en *streaming*, por lotes de `BATCH_SIZE` registros, y escribe en MongoDB y Neo4j con el mismo identificador de evaluación para mantener ambas bases consistentes.

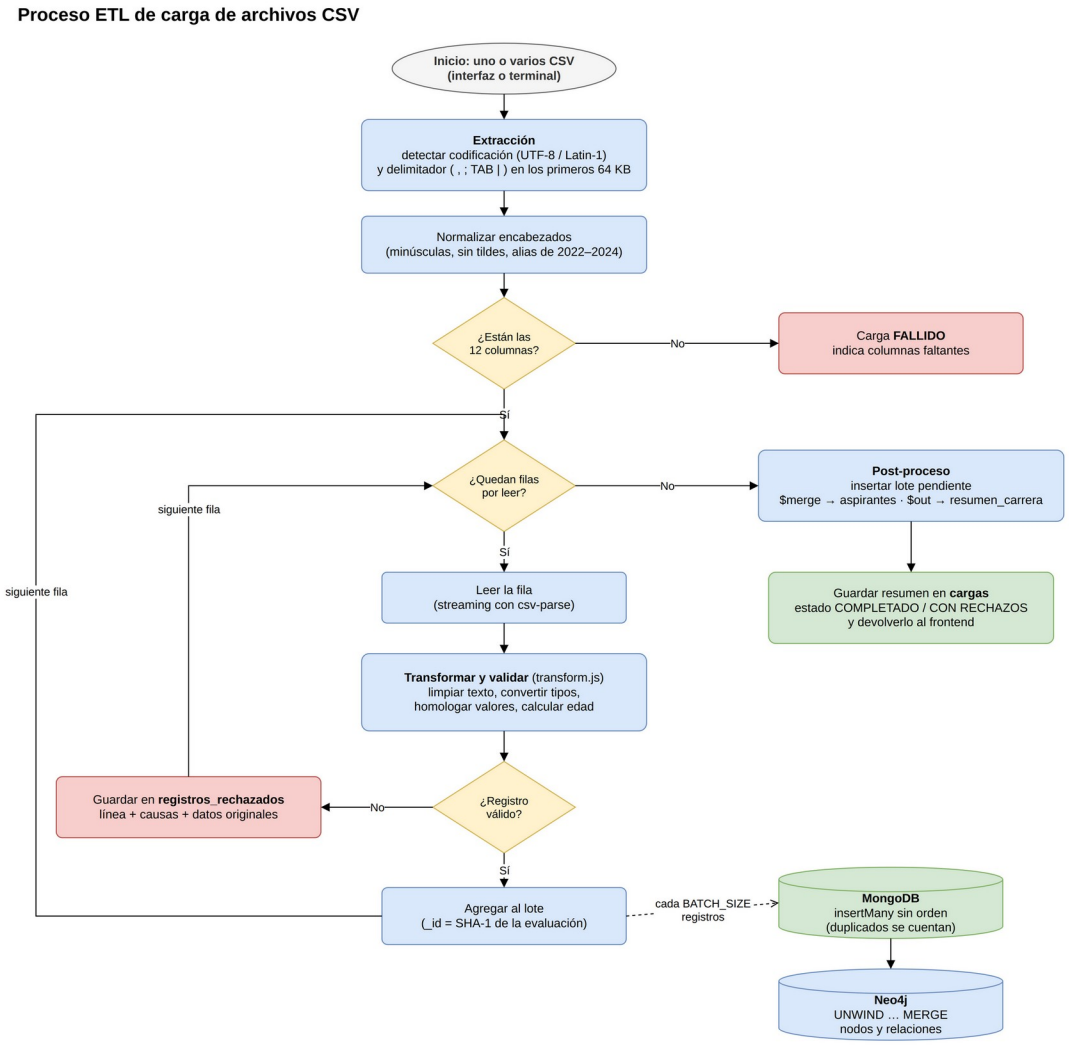


Figura 5. *Diagrama de flujo del proceso ETL*

5.1 Etapas

1. **Extracción:** se detecta la codificación (UTF-8 o Latin-1) y el delimitador (`,` `;` tabulador o `|`) leyendo los primeros 64 KB; se eliminan BOM y líneas vacías.
2. **Normalización de encabezados:** minúsculas, sin tildes y _ en lugar de espacios. Las columnas se identifican por nombre, no por posición.
3. **Validación de estructura:** si falta alguna de las 12 columnas, la carga termina con estado `FALLIDO` e indica cuáles faltan.

4. **Transformación y validación de cada registro** con las reglas de la sección 5.2.
5. **Carga por lotes:** `insertMany` sin orden en MongoDB (los duplicados se cuentan pero no detienen la carga) y `UNWIND … MERGE` en Neo4j.
6. **Post-proceso:** se regeneran las colecciones `aspirantes` y `resumen_carrera`.
7. **Reporte:** se guarda el resumen en `cargas` y se devuelve al frontend.

5.2 Reglas de limpieza y estandarización

Todos los campos de texto se recortan, se colapsan los espacios múltiples, se pasan a mayúsculas y se les quitan las tildes (la Ñ se conserva). Se consideran vacíos los valores `""`, `NULL`, `NA`, `N/A`, `NAN`, `NONE`, `-` y `SIN REGISTRO`.

Tabla 9. Reglas del ETL

Campo	Regla	Si no cumple
correlativo_aspirante	Obligatorio, alfanumérico; se guarda en minúsculas	**Rechazo**
fecha_asignacion	`AAAA-MM-DD` o `DD/MM/AAAA`; se valida que la fecha exista	**Rechazo**
materia	Obligatoria	**Rechazo**
numero_de_fecha_de_evaluacion	Entero ≥ 1 (acepta `2.0`); se guarda como `intento`	**Rechazo**
anio_de_ingreso	Entero entre 1990 y el año actual + 2	**Rechazo**
aprobacion	APROBADO/APROBADA → APROBADO; NO APROBADO/NO APROBADA/REPROBADO → NO APROBADO; se agrega el booleano `aprobado`	**Rechazo**
sexo	M/MASCULINO/HOMBRE → M; F/FEMENINO/MUJER → F	SIN REGISTRO (advertencia)
anio_nacimiento	Entero entre 1930 y el año actual; SIN REGISTRO → `null`	`null` (advertencia)
edad (derivada)	año de `fecha_asignacion` − `anio_nacimiento`, válida entre 12 y 80	`null` (advertencia)
carrera_objetivo	Vacío → NINGUNO; NINGUNO se conserva como categoría	—
departamento, municipio	Vacío → SIN REGISTRO	—
tipo_institucion_educativa	OFICIAL/PUBLICA → PUBLICO; PRIVADA → PRIVADO; vacío → SIN REGISTRO	—
Fila con otro número de columnas o comillas mal formadas	—	**Rechazo**
Registro repetido (mismo `_id`)	Ya cargado o repetido dentro del mismo archivo	Se cuenta como **duplicado**

Justificación: sólo se rechazan los registros a los que les falta información indispensable para identificar la evaluación o su resultado. Los registros con datos demográficos incompletos se conservan, porque siguen siendo útiles para las consultas que no usan la edad; las consultas de edad simplemente excluyen los valores `null`.

5.3 Particularidades de los archivos 2022–2024

Los archivos de 2022 a 2024 tienen las columnas en otro orden y algunos encabezados y valores recortados. El ETL los trata así:

Tabla 10. Tratamiento de los archivos 2022–2024

Situación	Tratamiento
Encabezados recortados: `numero_de_fecha_de_evaluaci`, `departamento_institucion_ed`, `municipio_institucion_`	Se traducen a su nombre completo con una tabla de alias (`ALIAS_ENCABEZADOS`)
Carrera `INGENIERIA EN CIENCIAS Y`	Se completa como `INGENIERIA EN CIENCIAS Y SISTEMAS` (no hay otra carrera que empiece así)
Carrera `INGENIERIA MECANICA` con espacio al final	Es el inicio de Mecánica Eléctrica o Mecánica Industrial; como no se puede saber cuál, se guarda como `INGENIERIA MECANICA (ELECTRICA O INDUSTRIAL)` para no inflar la carrera Mecánica
Correlativo de 20 a 23 caracteres (en lugar de 32)	Se conserva; sigue siendo único dentro de esos años, pero no se puede relacionar con años anteriores
Municipios recortados (p. ej. `GUATEMALA - CIUDAD`)	Se cargan tal como vienen, porque no es posible saber a qué zona corresponden

5.4 Resultado de la carga del dataset

Tabla 11. Carga de los archivos 2015–2024

Archivo	Leídos	Insertados	Duplicados	Rechazados	Tiempo	Estado
pruebas_especificas_2015.csv	20,277	20,274	3	0	2.81 s	COMPLETADO
pruebas_especificas_2016.csv	22,137	21,987	150	0	2.45 s	COMPLETADO
pruebas_especificas_2017.csv	21,581	21,399	182	0	2.70 s	COMPLETADO
pruebas_especificas_2018.csv	20,871	20,680	191	0	2.87 s	COMPLETADO
pruebas_especificas_2019.csv	17,865	17,835	30	0	3.06 s	COMPLETADO
pruebas_especificas_2020.csv	17,931	17,665	266	0	3.18 s	COMPLETADO
pruebas_especificas_2021.csv	30,644	30,644	0	0	4.79 s	COMPLETADO
pruebas_especificas_2022.csv	24,237	24,231	6	0	4.36 s	COMPLETADO
pruebas_especificas_2023.csv	19,047	19,047	0	0	4.27 s	COMPLETADO
pruebas_especificas_2024.csv	13,863	13,857	6	0	3.98 s	COMPLETADO

Archivo	Leídos	Insertados	Duplicados	Rechazados	Tiempo	Estado
Total	**208,453**	**207,619**	**834**	**0**	**34.5 s**	

Los 834 duplicados son filas idénticas repetidas dentro de los propios archivos de origen (por ejemplo, 150 en 2016 y 266 en 2020); el `_id` determinístico evita contarlas dos veces. Se registraron 4 advertencias por año de nacimiento fuera de rango, que se conservaron con `edad = null`.

6. Modelo de grafos (Neo4j)

El grafo representa las relaciones del dataset que en MongoDB requerirían varios `$lookup`. Cada aspirante se conecta con las materias que evaluó, la carrera a la que aspira y la institución de la que proviene; la institución se conecta con su tipo y su municipio, y el municipio con su departamento.

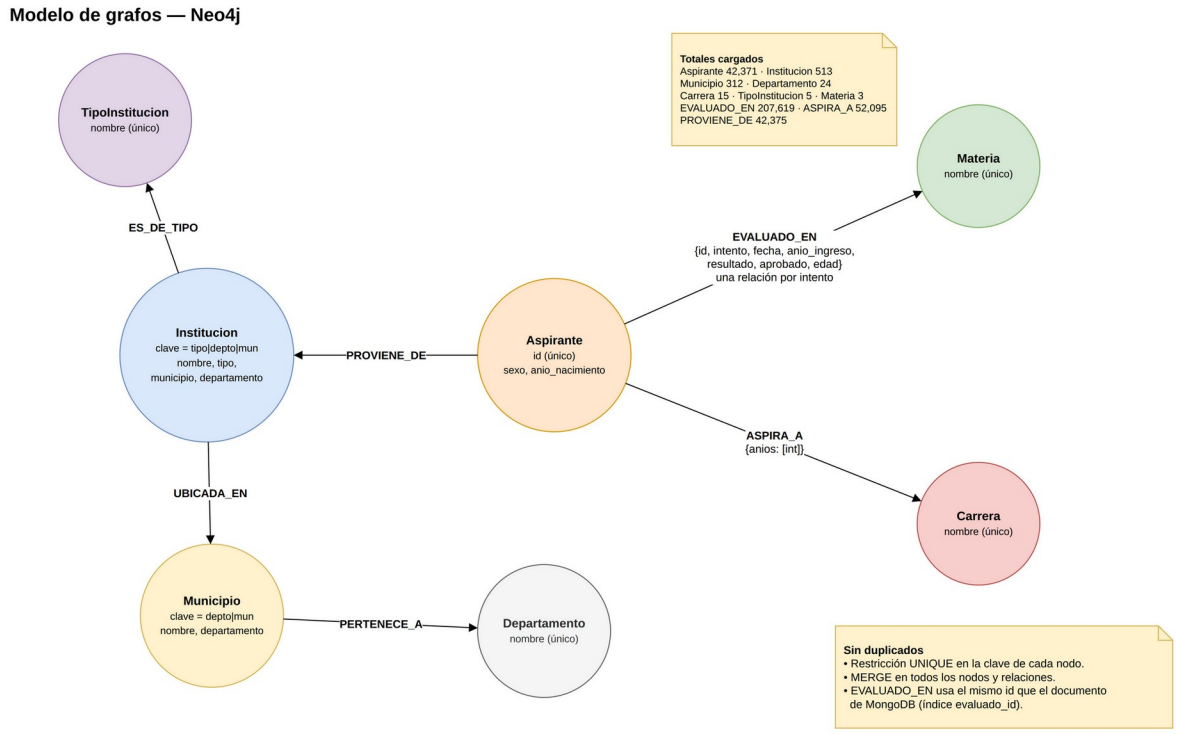


Figura 6. *Modelo de grafos de Neo4j*

Tabla 12. Nodos del grafo

Nodo	Clave única	Propiedades	Cantidad
Aspirante	`id` (correlativo)	sexo, anio_nacimiento	42,371
Materia	`nombre`	—	3
Carrera	`nombre`	—	15
Institucion	`clave` = tipo|departamento|municipio	nombre, tipo, municipio, departamento	513
TipoInstitucion	`nombre`	—	5
Municipio	`clave` = departamento|municipio	nombre, departamento	312
Departamento	`nombre`	—	24

Tabla 13. Relaciones del grafo

Relación	Origen → destino	Propiedades	Cantidad
EVALUADO_EN	Aspirante → Materia	id, intento, fecha, anio_ingreso, resultado, aprobado, edad	207,619
ASPIRA_A	Aspirante → Carrera	anios (lista de años de ingreso)	52,095
PROVIENE_DE	Aspirante → Institucion	—	42,375
ES_DE_TIPO	Institucion → TipoInstitucion	—	513
UBICADA_EN	Institucion → Municipio	—	513
PERTENECE_A	Municipio → Departamento	—	312

Decisiones de diseño:

- El dataset **no incluye el nombre del establecimiento**, sólo su tipo y ubicación. Por eso `Institucion` representa el grupo «tipo de institución en un municipio» (p. ej. `PRIVADO - QUETZALTENANGO`).
- `Municipio` usa una clave compuesta con el departamento para no confundir municipios con el mismo nombre en distintos departamentos.
- Cada intento es una relación `EVALUADO_EN` distinta, identificada con el mismo `id` del documento de MongoDB; así el grafo conserva el historial y permite filtrar por fecha, intento o resultado.
- **Sin duplicados:** restricciones `UNIQUE` en la clave de cada nodo, índice sobre `EVALUADO_EN.id` y `MERGE` en todos los nodos y relaciones. Se verificó que no existen aspirantes duplicados (consulta de control con resultado 0).

7. API REST

Todas las rutas responden JSON. El frontend las consume a través de nginx en `http://localhost:5174/api`; el backend también está disponible directamente en `http://localhost:3001/api`.

Tabla 14. Rutas de la API

Método	Ruta	Descripción
GET	/api/salud	Estado de la API, MongoDB y Neo4j (503 si alguna base está caída)
POST	/api/cargas	Carga de uno o varios CSV (campo `archivos`, multipart)
GET	/api/cargas	Historial de cargas
GET	/api/cargas/:id/rechazados	Registros rechazados de una carga (`pagina`, `limite`)
DELETE	/api/cargas/datos?confirmar=si	Vacía MongoDB y Neo4j
GET	/api/mongo/consultas	Lista de las 20 consultas de MongoDB
GET	/api/mongo/consultas/:id	Ejecuta una consulta (`anio`, `aspirante`, `tipo`) y devuelve datos y pipeline
GET	/api/mongo/kpis	Indicadores generales (`anio` opcional)
POST	/api/mongo/consola	Pipeline libre de sólo lectura (`{coleccion, pipeline}`)
GET	/api/grafo/consultas	Lista de las consultas Cypher
GET	/api/grafo/consultas/:id	Ejecuta una consulta Cypher con sus parámetros
GET	/api/grafo/explorar	Subgrafo filtrado (carrera, departamento, municipio, tipo, materia, resultado, año, límite)
GET	/api/grafo/estadisticas	Cantidad de nodos y relaciones por tipo
POST	/api/grafo/consola	Cypher libre en transacción de sólo lectura (`{cypher}`)
GET	/api/busqueda	Buscador con filtros combinables y paginación
GET	/api/catalogos	Valores distintos para llenar los filtros

Formato de error común a todas las rutas:

```
{ "error": "mensaje legible en español", "detalle": "información adicional (opcional)" }
```

8. Manejo de errores

Ningún error detiene el servidor ni cierra la interfaz. El middleware `errors.js` convierte cada error en una respuesta JSON con un mensaje en español y el código HTTP adecuado; el frontend lo muestra en un recuadro de alerta en la sección afectada.

Tabla 15. Manejo de errores

Situación	Comportamiento
Archivo que no es CSV	400: «El archivo X no es un CSV»
CSV sin las columnas esperadas	La carga termina con estado `FALLIDO` e indica las columnas faltantes
Registro inválido	Se rechaza, se guarda con su causa y la carga continúa
MongoDB o Neo4j apagado	El servidor sigue en ejecución; las rutas afectadas responden 503 «No fue posible conectar con Neo4j/MongoDB» en unos 5 segundos y el indicador de la barra superior se pone en rojo. Al volver la base, la conexión se recupera sola
Parámetro inválido o faltante	400 con el nombre del parámetro
Mismo aspirante como origen y destino del camino más corto	400: «El aspirante origen y el aspirante destino deben ser diferentes»
Error de sintaxis en la consola	400: «Error de sintaxis en la consulta Cypher» o «Error en el pipeline», con la posición del error
Intento de escritura en la consola	Bloqueado: `$out`/`$merge` rechazados y Cypher en transacción de lectura («La consola sólo permite consultas de lectura»)
Error al dibujar un componente	`ErrorBoundary` muestra el error sólo en esa sección y permite reintentar
Backend apagado	El frontend muestra «No se pudo conectar con el servidor (backend)»

9. Ventajas y limitaciones de usar MongoDB y Neo4j

Tabla 16. Comparación de los dos motores

Aspecto	MongoDB	Neo4j
Fortalezas en este proyecto	Agregaciones estadísticas sobre 207 mil registros (`$match`, `$group`, `$project`, `$sort`), esquema flexible para datos semiestructurados, historial del aspirante embebido en un solo documento	Recorridos de varios saltos (departamento → municipio → institución → aspirante → carrera), caminos más cortos y filtros sobre propiedades de las relaciones
Limitaciones	Relacionar entidades requiere `$lookup` o duplicar datos; los recorridos de varios niveles son costosos	Las agregaciones masivas son más lentas; la carga con `MERGE` es más costosa que un `insertMany`
Costo de tener ambas	Los datos se duplican y deben mantenerse sincronizados	El ETL escribe en las dos bases con el mismo identificador de evaluación para garantizar la consistencia

Las consultas de cada motor y su documentación detallada están en el documento *Consultas MongoDB y Cypher*; el análisis de los resultados obtenidos con los datos reales está en el documento *Análisis de resultados y conclusiones*.